
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 13

O Pré-processador em C

Exercícios (13.4–13.10)

Macros com argumentos e suas aplicações práticas

Contents

1	Exercício 13.4 – Volume de Esfera (Macro)	2
2	Exercício 13.5 – Macro SUM	3
3	Exercício 13.6 – Macro MINIMUM2	3
4	Exercício 13.7 – Macro MINIMUM3	5
5	Exercício 13.8 – Macro PRINT	6
6	Exercício 13.9 – Macro PRINTARRAY	6
7	Exercício 13.10 – Macro SUMARRAY	8

1. Exercício 13.4 – Volume de Esfera (Macro)

Resposta detalhada

```
1  /* Ex13.4: volume_esfera.c
2     Macro com um argumento + tabela de raios 1 a 10 */
3  #include <stdio.h>
4  #define PI 3.14159
5  #define VOLUME_ESFERA( r ) ( ( 4.0 / 3.0 ) * PI * ( r
6     ) * ( r ) * ( r ) )
7
8  int main( void )
9  {
10
11     int raio;
12
13     printf( "%-8s%-15s\n", "Raio", "Volume" );
14     printf( "-----\n" );
15
16     for ( raio = 1; raio <= 10; ++raio ) {
17         printf( "%-8d%-15.4f\n", raio, VOLUME_ESFERA(
18             raio ) );
19     }
```

Saída esperada:

Saída do programa

```
Raio Volume
-----
1 4.1888
2 33.5103
3 113.0972
4 268.0826
5 523.5984
6 904.7782
7 1436.7551
8 2144.6624
9 3053.6334
10 4188.7867
```

Observação importante: note os parênteses em torno de (r) – sem eles, VOLUME_ESFERA(2+1) seria expandido para (4.0/3.0)*3.14159*2+1*2+1*2+1, que devido à precedência da multiplicação daria (4.0/3.0)*3.14159*2 + 1*2 + 1*2 + 1 = 8.378 + 5 = 13.378 (errado!).

2. Exercício 13.5 – Macro SUM

Resposta detalhada

```
1  /* Ex13.5: macro_sum.c */
2  #include <stdio.h>
3  #define SUM( x, y ) ( ( x ) + ( y ) )
4
5  int main( void )
6  {
7      int x = 4, y = 9;
8      printf( "A soma de x e y e %d\n", SUM( x, y ) );
9      return 0;
10 }
```

Saída:

Saída do programa

A soma de x e y e 13

Vantagens de macro vs. função:

- **Sem overhead de chamada** – a expansão ocorre em tempo de compilação.
- **Funciona com qualquer tipo** – SUM(2.5, 3.1) também funciona.

Desvantagens:

- **Sem checagem de tipos** – erros podem passar despercebidos.
- **Avaliação múltipla** – argumentos podem ser avaliados várias vezes (perigoso com efeitos colaterais como SUM(i++, j++)).
- **Difícil de depurar** – o depurador não consegue entrar em macros.

3. Exercício 13.6 – Macro MINIMUM2

Resposta detalhada

```
1  /* Ex13.6: minimum2.c */
2  #include <stdio.h>
3  #define MINIMUM2( a, b ) ( ( a ) < ( b ) ? ( a ) : ( b ) )
4
5  int main( void )
6  {
7      double x, y;
8
9      printf( "Digite dois valores: " );
10     scanf( "%lf%lf", &x, &y );
11
12     printf( "O menor e %.2f\n", MINIMUM2( x, y ) );
13     return 0;
```

14 }
|

Conceito: usamos o operador ternário `? : –` expressão que retorna um valor, perfeita para macros (diferente de `if-else`, que é instrução).

Erro comum de programação

Cuidado com efeitos colaterais:

```
1 int i = 5, j = 10;  
2 int m = MINIMUM2( ++i, ++j );  
3 /* expande para: ((++i) < (++j) ? (++i) : (++j))  
4    i sera incrementado DUAS vezes (uma na  
5    comparacao,  
    outra no retorno) -- comportamento inesperado!  
    */
```

Em funções reais, os argumentos são avaliados uma única vez.

4. Exercício 13.7 – Macro MINIMUM3

Resposta detalhada

```

1  /* Ex13.7: minimum3.c
2     Macro definida em termos de MINIMUM2 */
3  #include <stdio.h>
4  #define MINIMUM2( a, b )      ( ( a ) < ( b ) ? ( a ) :
5     ( b ) )
6
7  #define MINIMUM3( a, b, c ) ( MINIMUM2( MINIMUM2( a, b
8     ), c ) )
9
10 int main( void )
11 {
12     double x, y, z;
13
14     printf( "Digite tres valores: " );
15     scanf( "%lf%lf%lf", &x, &y, &z );
16
17     printf( "O menor e %.2f\n", MINIMUM3( x, y, z ) );
18     return 0;
19 }

```

Expansão passo a passo de MINIMUM3(5, 3, 8):

- a) MINIMUM2(MINIMUM2(5, 3), 8)
- b) MINIMUM2((5 < 3 ? 5 : 3), 8)
- c) ((5 < 3 ? 5 : 3) < 8 ? (5 < 3 ? 5 : 3) : 8)
- d) Avaliação interna: 5 < 3 é falso → retorna 3.
- e) (3 < 8 ? 3 : 8) → 3.

Lição: macros podem ser *composicionais* – definir uma em termos de outra. Esse é o princípio por trás de bibliotecas de macros como #define MAX(a, b) e #define MAX3(a, b, c) MAX(MAX(a, b), c).

Boa prática de programação

Note que MINIMUM3(++i, ++j, ++k) expandiria ++i e ++j *quatro vezes* – pelo aninhamento das macros. **Nunca passe expressões com efeitos colaterais para macros.**

5. Exercício 13.8 – Macro PRINT

Resposta detalhada

```
1  /* Ex13.8: macro_print.c */
2  #include <stdio.h>
3  #define PRINT( s ) printf( "%s\n", s )
4
5  int main( void )
6  {
7      PRINT( "Ola, mundo!" );
8      PRINT( "Macros sao uteis." );
9      PRINT( "Fim do programa." );
10
11     return 0;
12 }
```

Saída:

Saída do programa

```
Ola, mundo!
Macros sao uteis.
Fim do programa.
```

Variação útil: macro de log com origem usando `__FILE__` e `__LINE__`:

```
1  #define LOG( s ) printf( "[%s:%d] %s\n", __FILE__,
2      __LINE__, s )
3
4  LOG( "Iniciando processamento" );
5  /* Saída: [exemplo.c:8] Iniciando processamento */
```

Esse é o padrão básico de *logging frameworks* em C.

6. Exercício 13.9 – Macro PRINTARRAY

Resposta detalhada

```
1  /* Ex13.9: macro_print_array.c */
2  #include <stdio.h>
3  #define PRINTARRAY( arr, n )
4      do {
5          int _i;
6          for ( _i = 0; _i < ( n ); ++_i ) {
7              printf( "%d ", ( arr )[ _i ] );
8          }
9          printf( "\n" );
10     } while ( 0 )
11
12 int main( void )
```

```
\
\
\
\
\
\
```

```
13 {
14     int valores[] = { 10, 20, 30, 40, 50, 60 };
15     int n = sizeof( valores ) / sizeof( valores[ 0 ] )
16         ;
17     printf( "Array: " );
18     PRINTARRAY( valores, n );
19
20     return 0;
21 }
```

Truques importantes desta macro:

- **\ no fim das linhas:** continuação da macro (do contrário, terminaria na primeira quebra).
- **do { ... } while (0):** idioma para encapsular múltiplas instruções em um bloco que pode ser usado como instrução única (com ;).
- **_i em vez de i:** convenção para evitar conflito com variáveis do usuário (mas macros não têm escopo próprio – cuidado!).

Por que do...while(0)? Para que a macro se comporte como uma instrução simples mesmo em `if` sem chaves:

```
1 if ( cond )
2     PRINTARRAY( a, n ); /* expande corretamente */
3 else
4     outraInstrucao();
```

7. Exercício 13.10 – Macro SUMARRAY

Resposta detalhada

```

1  /* Ex13.10: macro_sum_array.c */
2  #include <stdio.h>
3  #define SUMARRAY( arr, n, total )           \
4      do {                                   \
5          int _i;                             \
6          ( total ) = 0;                       \
7          for ( _i = 0; _i < ( n ); ++_i ) {   \
8              ( total ) += ( arr )[ _i ];      \
9          }                                     \
10     } while ( 0 )
11
12 int main( void )
13 {
14     int valores[] = { 10, 20, 30, 40, 50 };
15     int n = sizeof( valores ) / sizeof( valores[ 0 ] )
16         ;
17     int total;
18
19     SUMARRAY( valores, n, total );
20
21     printf( "Soma: %d\n", total );
22     return 0;
23 }

```

Saída:

Saída do programa

Soma: 150

Comparação macro vs. função:

Versão função (mais segura):

```

1  int sumArray( const int *arr, int n )
2  {
3      int i, total = 0;
4      for ( i = 0; i < n; ++i ) total += arr[ i ];
5      return total;
6  }
7
8  /* Uso */
9  int total = sumArray( valores, n );

```

Quando preferir macro a função?

- **Performance crítica:** código inserido em linha, sem overhead.
- **Tipo genérico:** SUMARRAY funciona com int, double, long... (função precisaria de variantes).

- **Acesso a contexto local:** macros podem usar variáveis do contexto onde são expandidas.

Quando preferir função?

- **Tamanho importa:** cada uso de macro adiciona código; cada uso de função é só uma chamada.
- **Recursão:** macros não podem recursão (auto-referência é detectada e bloqueada).
- **Tipagem:** compilador valida tipos em funções; em macros, não.
- **Depuração:** função aparece na pilha de chamadas; macro é “transparente”.

Boa prática de programação

Em projetos modernos, prefira `inline` (C99+) ou `static inline` para pequenas funções. Você ganha o melhor dos dois mundos: tipagem checada + expansão em linha (a critério do compilador).

Reflexão Final – Capítulo 13

O pré-processador: ferramenta de duas faces

O pré-processador é uma ferramenta dupla: poderosa e perigosa.

Onde brilha:

- **Constantes simbólicas:** nomes legíveis para valores literais.
- **Compilação condicional:** código portátil entre Linux, Windows, macOS via `#ifdef __linux__`, etc.
- **Guardas de cabeçalho:** `#ifndef MEU_H` previne inclusão múltipla.
- **Macros de debug:** `assert`, `LOG`, com `__FILE__` e `__LINE__`.
- **Geração de código repetitivo:** X-macros, `FOREACH`, etc.

Onde dá problema:

- **Falta de tipagem:** erros que apareceriam em compilação passam para tempo de execução.
- **Avaliação múltipla:** `MAX(f(), g())` chama `f` duas vezes.
- **Não respeita escopo:** macros são puramente textuais.
- **Erros confusos:** compiladores reportam erros no *código expandido*, não no original.

Substitutos modernos:

- `const` em vez de `#define` para constantes.
- `enum` para conjuntos de constantes relacionadas.
- `inline` (C99+) em vez de macros como pequenas funções.
- `_Generic` (C11) para genéricos sem macros complexas.
- Templates em C++ (mas C puro ainda usa muito macro).

Próximo capítulo: Cap. 14 traz tópicos avançados que tecem tudo o que aprendemos: `va_list` (funções com número variável de argumentos), `atexit`, manipulação de sinais, argumentos da linha de comando, e mais. É a fronteira final entre seu conhecimento atual e a maturidade total em C.